

MSK Pulse

Project Documentation

Operational analytics and a prior-authorization denial-risk model for a simulated MSK clinic network

1. Executive Summary

MSK Pulse is a self-contained data science project that pairs open-ended operational analytics with a production-style machine learning service, built on top of one shared, versioned data warehouse. It simulates a 12-clinic musculoskeletal (MSK) provider network — referrals, intake, prior authorizations, visits, physical therapy adherence, and patient engagement over a 24-month period — and answers the kind of questions an operations- or product-facing data science function is expected to own: where patients drop off in the journey, how retention differs across cohorts and payers, whether a product change actually moved a metric, why a data-quality check failed, and why one clinic underperforms its peers.

The single largest, most controllable finding — that prior-authorization denial is a bigger drop-off point than intake, and one that can be addressed proactively rather than through patient behavior change — became the basis for the second half of the project: a calibrated, explainable machine learning model that flags high-risk prior-authorization submissions before they are sent, served through a live API and surfaced directly in an operations dashboard.

Every component in this repository — the data, the warehouse, the metrics definitions, the analyses, the model, the API, and the dashboard — is built to the same standard: reproducible from a clean checkout, tested, and documented, rather than a one-off notebook exercise.

2. Business Purpose

Prior-authorization denials are one of the most expensive, most avoidable sources of friction in a specialty care operation. Every denied authorization means a delayed or abandoned episode of care, additional administrative rework to appeal or resubmit, and a patient who has already dropped out of the funnel by the time the appeal succeeds, if it succeeds at all. For an MSK-focused provider network in particular — where prior authorization gates almost every high-value procedure and course of physical therapy — the ability to see this problem coming, rather than reacting to it after a denial letter arrives, is a direct lever on both patient outcomes and operating margin.

This project is built around that specific business problem, and structured the way a real analytics/ML function inside such an organization would need to structure it, in three layers:

- Understand the problem quantitatively first. Before building anything predictive, the funnel, cohort, and clinic-level analyses establish where the actual drop-off is happening, so that the modeling effort is aimed at the highest-leverage problem rather than an interesting but low-impact one.

- Make measurement trustworthy. A metrics dictionary, warehouse tests, and a documented data-quality monitor exist so that a number quoted in one place (a dashboard) means the same thing as the same number quoted somewhere else (an analysis memo, a model feature) — a common failure point in real organizations that this project deliberately does not reproduce.
- Act on the finding, in production. The denial-risk model is not a notebook exercise sitting next to the analytics — it is served behind a live API and surfaced in the same dashboard operations staff would already be looking at, so the finding from the analysis phase becomes something a real workflow can use.

Framed this way, MSK Pulse is a demonstration of the full life cycle a health-tech operations & analytics function is responsible for: diagnose with data, quantify the opportunity, build the intervention, and ship it somewhere it can actually be used — not just three separate skills shown in isolation.

3. What This Project Delivers

The repository is organized as seven components, each independently useful and independently testable, that compose into one pipeline.

Component	What it is	Business question it answers
Synthetic data generator	A seeded generator producing a realistic 12-clinic MSK network over 24 months, with three intentionally planted scenarios (a payer feed data-quality break, a feature launch, and an underperforming clinic).	Provides a safe, reproducible substitute for real claims/EHR data with genuine, discoverable patterns.
DuckDB + dbt warehouse	A staging → marts warehouse with data tests, built and validated on every push.	Where does a trustworthy number come from, and how do we know it's still correct after a change?
Metrics dictionary	Metric definitions maintained once, in YAML, and rendered to Markdown documentation.	Does "approval rate" mean the same thing on every dashboard and in every analysis?
Five analyses	Funnel, cohort/retention, feature-launch measurement (diff-in-diff + CUPED), a data-quality anomaly postmortem, and an open-ended clinic investigation.	Where do patients drop off, did a launch work, why did a metric break, and why does one clinic underperform?
Denial-risk model	A calibrated, SHAP-explained XGBoost classifier trained on a time-based split of the warehouse.	Can we flag a prior-authorization submission's denial risk before it is sent?
Scoring API	A live Gradio Space (with an equivalent FastAPI + Docker service also included) serving the trained model.	How does this prediction reach a real workflow, not just a notebook?
Ops dashboard	A four-page Streamlit application — executive overview, funnel, cohorts & retention, and a live prior-authorization risk screen calling the deployed API.	Where does a non-technical operations user actually see and act on this?

A GitHub Actions workflow rebuilds all seven components from a clean checkout on every push — regenerating the data, rebuilding the warehouse, rendering the metrics dictionary, running all five analyses, retraining the model, and executing the API test suite — so the whole pipeline's reproducibility is checked automatically rather than asserted.

4. Architecture

The diagram below shows the full pipeline: a data foundation feeding both the analytics layer and, indirectly, the model; the analyses converging into the single highest-leverage modeling problem they identified; and the trained model reaching two different front doors — a live scoring API and an operations dashboard that calls it.

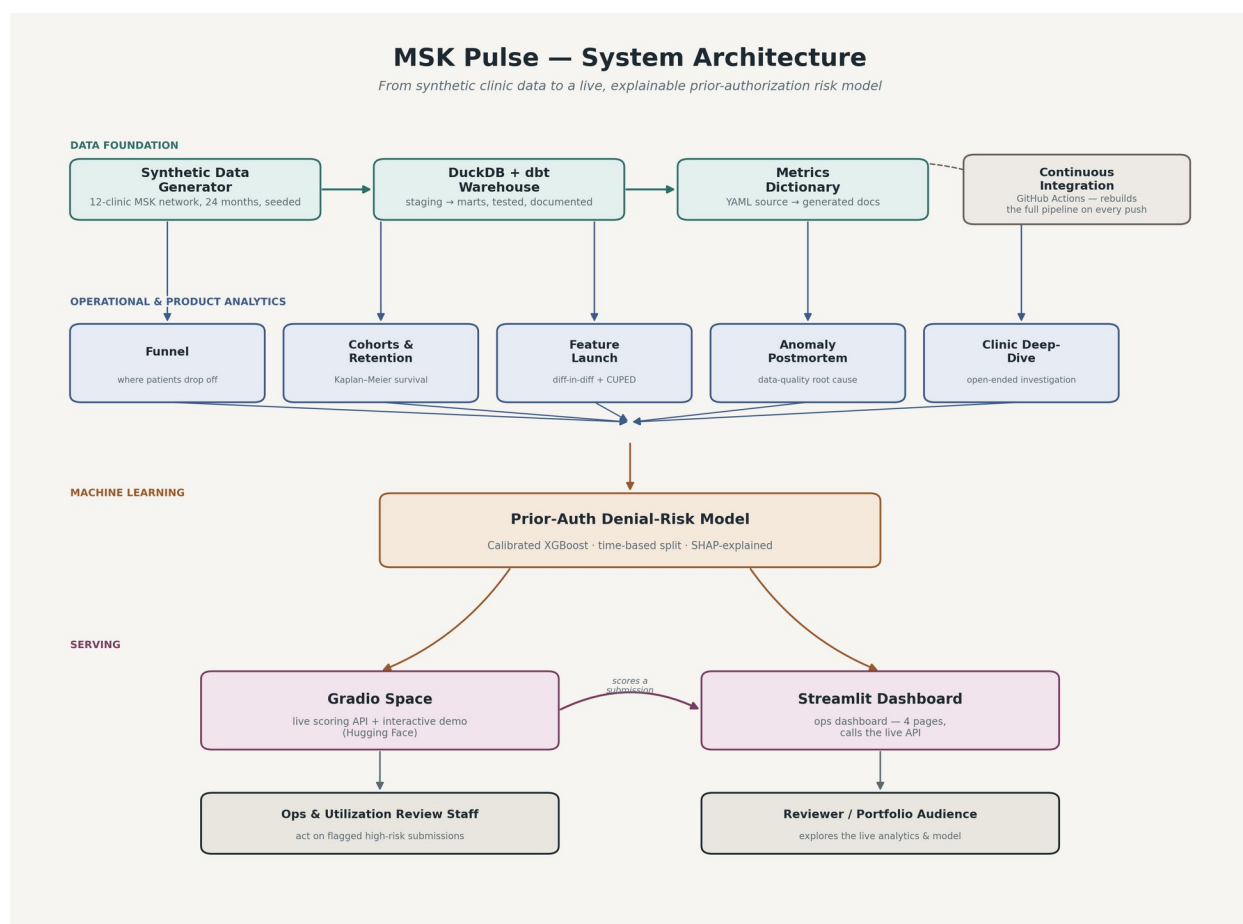


Figure 1. End-to-end system architecture, from synthetic data to the live scoring service.

Two design choices are worth calling out explicitly. First, the warehouse is the single source of truth for both the analyses and the model's training data — there is no separate, divergent "model dataset" assembled outside the warehouse, which is a common source of analytics/ML inconsistency in real organizations. Second, the API and dashboard are genuinely two separate deployable services communicating over HTTP, the same way a real internal tool would call out to an independently deployed model service, rather than one monolithic application.

5. Technology Choices, and the Alternatives Considered

Every tool in this stack was chosen for a specific reason tied to the problem at hand, not by default. The table below documents the decision and what else was considered.

Layer	Chosen	Why	Alternatives considered
Warehouse	DuckDB + dbt	An embedded, file-based warehouse with a real dbt project (staging/marts, tests, docs) gives the exact same modeling discipline as a cloud warehouse (Snowflake/BigQuery/Redshift) without needing a hosted database, which matters for a project meant to run from a clean checkout in CI.	A hosted Postgres or Snowflake trial; plain pandas/SQL scripts without dbt. Both were rejected — a hosted database adds a dependency CI can't easily provision, and skipping dbt would mean no lineage, no tests, and no documentation generation.
Survival analysis	Hand-implemented Kaplan–Meier estimator	The 'lifelines' package failed to build in the target environment; rather than blocking on that dependency, the estimator was implemented directly against its well-defined statistical formula.	The 'lifelines' library (standard choice, but blocked by a build failure) and a simpler retention-curve approximation (rejected as statistically weaker).
Feature-launch measurement	Difference-in-differences with CUPED variance reduction	A naive before/after comparison cannot separate a launch's effect from a concurrent trend; diff-in-diff isolates it, and CUPED was added to reduce variance, with honest reporting when it underperforms due to a small cluster count rather than hiding that limitation.	A simple pre/post comparison (rejected — confounds trend and effect) and a full randomized experiment framework (out of scope for a retrospective analysis on existing data).
Model	XGBoost, probability-calibrated with isotonic regression	Gradient-boosted trees handle the mixed categorical/numeric feature set (payer, CPT/ICD codes, clinic quality, timing) well with minimal preprocessing, and isotonic calibration turns its raw scores into probabilities that are actually close to true empirical rates — important when a "22% risk" number is shown directly to an operations user.	Logistic regression (more interpretable by default, but weaker on non-linear interactions this data has), LightGBM (comparable performance, XGBoost's tooling and SHAP support were more mature for this use case), and an uncalibrated model (rejected — raw gradient-boosted probabilities are not well-calibrated out of the box).
Explainability	SHAP TreeExplainer, with a feature-importance fallback	Per-prediction SHAP values let an operations user see which specific factors are pushing a specific submission's risk up or down, not just which features matter on average.	LIME (comparable purpose, weaker fit for tree models) and permutation importance alone (only global, not per-prediction — insufficient for a UI meant to explain one specific submission).
Experiment tracking	MLflow, SQLite backend	Local, file-based experiment tracking that still gives a full run history, without requiring a hosted tracking server.	Weights & Biases (requires an external account/service) and no tracking at all (rejected — makes model iteration

Layer	Chosen	Why	Alternatives considered
			unauditable).
Scoring API	Gradio Space (primary), FastAPI + Docker (included as an alternative)	A Gradio app exposes both an interactive demo page and a callable API from one file, with no container build step — the fastest path to something a reviewer can actually click on and a real client can actually call. The FastAPI/Docker service was built first and is fully tested; it remains in the repo as the more "production-shaped" option for any Docker-capable host.	A bare Flask/FastAPI app with no UI (rejected as the sole option — a portfolio audience benefits from something clickable, not just an OpenAPI spec) and a fully custom frontend (unnecessary engineering effort for the value it would add).
Dashboard	Streamlit	Fastest path from a pandas/DuckDB query to an interactive, multi-page internal tool, with first-class support for exactly this kind of ops-facing use case.	Dash/Plotly (more customizable, slower to build with) and a custom React frontend (far more engineering effort than the audience or use case justifies).
CI	GitHub Actions	Free, natively integrated with the repository host, sufficient for a linear pipeline of lint → build → test steps.	Jenkins or CircleCI (both add operational overhead disproportionate to this project's scope).

6. Challenges Encountered, and How They Were Resolved

Building and deploying this project surfaced a series of real engineering problems, several of which are common failure modes in applied ML work and worth documenting on their own merits.

6.1 Cross-platform development (Windows)

The project was developed and iterated on in a Windows environment, which surfaced encoding and tooling issues that don't appear on Unix-like systems: Windows' default ``cp1252`` text encoding cannot represent characters such as an em dash or arrow used in generated report text, causing ``UnicodeEncodeError`` failures on writes that worked fine elsewhere. This was resolved by making the target encoding explicit (``encoding="utf-8"``) on every file write across the codebase, rather than relying on the platform default. Separately, the absence of a ``make`` utility on Windows meant every pipeline stage needed a documented direct-command path, not just a ``Makefile`` target — which is why the project's documentation gives every step as an explicit command rather than assuming a Unix-style build tool is available.

6.2 Deployment platform constraints

The original deployment plan served the scoring model as a containerized FastAPI service. Partway through deployment, it became clear that Docker-based Hugging Face Spaces require a paid account tier that was not available. Rather than treat this as a blocker, the API deployment was re-architected around the Gradio SDK, which Hugging Face Spaces support without a container build — the FastAPI/Docker service was kept in the repository as a fully tested alternative for any environment where Docker hosting is available, rather than being discarded.

A related platform constraint appeared next: the account's free Gradio Spaces default to on-demand "ZeroGPU" hardware, which requires at least one function in the app to be explicitly decorated for GPU allocation before the Space will start — even though this model is small enough not to need a GPU at all. The fix was a decorator that requests GPU allocation when the ``spaces`` package is present (i.e., when actually running on a Space) and silently no-ops otherwise (i.e., when running locally), so the same file works correctly in both environments.

6.3 A version deadlock between the model library and its explainability library

The most involved problem encountered was a dependency deadlock between XGBoost's internal model serialization format and the version of SHAP available on the deployment platform's fixed Python version. Newer XGBoost releases changed how they serialize a trained model's internal ``base_score`` value; the SHAP version compatible with that newer format requires a newer Python than the deployment platform's base image provided, and the deployment platform's Python version could not be changed through the most obvious configuration path. Several narrower fixes were attempted and each hit its own version ceiling (pinning only scikit-learn, pinning only XGBoost, pinning only SHAP), before two changes together resolved it: pinning the exact mutually-compatible trio of scikit-learn, XGBoost, and SHAP versions, and forcing the platform to rebuild against a newer Python version with an explicit configuration change plus a full clean rebuild (a "factory reboot") rather than an incremental one, which is what had silently prevented an earlier attempt at the same fix from taking effect.

Independently of that fix, the service was also hardened against the underlying failure mode: the model-loading code now catches an explainability-library initialization failure and falls back to the model's built-in global feature importances rather than crashing the entire service, so a future platform-level version mismatch degrades the quality of the explanation shown to a user instead of taking the API down.

6.4 A silent API contract mismatch

After deployment, the scoring API's demo page worked correctly, but a direct client call to the documented ``/score`` endpoint failed. The root cause was a default in the Gradio framework itself: an app's callable API endpoint is not automatically named after its underlying function, and defaults to a generic name unless explicitly set. This was invisible from the interactive demo page, because the UI calls the function directly rather than through the named endpoint. The fix was to explicitly name the endpoint to match the contract already documented for API consumers, and the episode is a useful reminder that a UI working correctly is not sufficient evidence that an API contract is being met — the two need to be tested separately, which is now called out directly in this project's own test suite going forward.

6.5 Configuration drift between two deployment targets

The scoring API and the dashboard are deployed to two different platforms from two related but separate local copies of the code. At one point, a fix made to the API deployment folder was not correspondingly pushed to the main GitHub repository that the dashboard's platform builds from, so the dashboard's deployed copy silently continued pointing at a local address that no longer existed once the API moved off it. This was diagnosed by directly diffing the deployed branch against the working local copy rather than assuming they matched, and is the reason this project's documentation is explicit about exactly which files need to be kept in sync across which deployment targets.

6.6 Lint and formatting drift

Once continuous integration was in place, it caught two categories of issue that had not surfaced in earlier manual runs: an unsorted import block, and files missing a trailing newline at end of file (a common side effect of editing on Windows, where some editors do not add one back on save). Both were fixed directly, and the CI pipeline now exists specifically to catch this class of small, easy-to-miss issue automatically on every future change rather than relying on it being noticed by inspection.

7. Notable Design Decisions

- Time-based train/calibration/test split. The model is evaluated on data that comes strictly after its training window, rather than a random split, which would let future information leak into training and produce an artificially optimistic evaluation.
- Deliberately honest model performance. The synthetic data generator was tuned to give the model genuine, learnable signal without inflating it artificially — the reported ROC-AUC and PR-AUC reflect a realistic, defensible level of predictive power for this kind of problem, not a maximized number. A model that looks too good on synthetic data is a weaker portfolio signal than one whose performance is plausible and whose limitations are documented.
- Warn-severity data-quality tests, not just pass/fail. The dbt data-quality test that catches the planted payer feed break is deliberately configured to warn rather than hard-fail the build, mirroring how a real data quality monitor should surface a problem for investigation without blocking every other pipeline consumer.
- Graceful degradation over hard failure in the serving layer. The explainability fallback described in Section 6.3 reflects a general principle applied throughout the API: a secondary feature (per-prediction explanation) failing should degrade the response, not take down the primary feature (the risk score itself).
- Documentation and reproducibility as first-class deliverables, not an afterthought. The metrics dictionary, model card, CI pipeline, and this document all exist because a project's value in a professional context depends on someone else being able to understand, trust, and rebuild it — not only on the code running once.

8. Current Limitations

- All data is synthetic. Approval and utilization base rates are loosely calibrated to public claims statistics, but the specific causal relationships in the generator are authored assumptions, not fitted to real claims data — the analyses and model demonstrate method, not a validated real-world finding.
- The model is trained and evaluated on a single synthetic cohort/time window; it has not been tested for performance drift over a longer horizon or validated against a genuinely held-out population.
- The scoring API currently exposes prediction and explanation only — it does not yet log predictions for later audit, monitor for input or prediction drift in production, or expose confidence intervals around the calibrated probability.

- The dashboard has no authentication layer; as deployed, it is intended as a public portfolio demonstration rather than a tool handling real patient-level data.

9. Future Roadmap

The project is deliberately scoped to be complete and coherent as it stands, but several extensions would meaningfully increase its depth and its realism as a production system.

9.1 Model and ML infrastructure

- Add a monitoring layer that tracks prediction and feature drift over time against a reference window, with alerting when the live input distribution departs from the training distribution.
- Log every scored submission (prediction, inputs, and eventual outcome once known) to build a feedback loop for periodic retraining and for a proper offline evaluation of real-world calibration drift.
- Extend the model from a binary approve/deny risk score to a multi-class or multi-label formulation that also predicts the likely denial reason, which is more directly actionable for the staff member preparing the submission.
- Add a cost-sensitivity analysis to the threshold selection — the current threshold is chosen from the calibration data alone; a real deployment should weigh the cost of a missed denial against the cost of an unnecessary manual review.
- Compare XGBoost against at least one additional model family (e.g. a regularized logistic regression baseline and a LightGBM challenger) with a documented model-selection rationale, rather than a single model chosen at the outset.

9.2 Data and warehouse

- Introduce a proper feature store layer between the warehouse and the model, so training and serving-time feature computation are guaranteed to match rather than independently re-implemented (as they currently are, in the model's training pipeline and the API's request-time feature construction).
- Add more dbt data-quality tests covering referential integrity and distributional checks across all marts, not only the one planted anomaly scenario.
- Move the warehouse from DuckDB to a hosted warehouse (Snowflake, BigQuery, or a managed Postgres) as a documented "scaling up" path, with the dbt project itself needing minimal change given dbt's warehouse-agnostic design.

9.3 Serving and application layer

- Add authentication and role-based access to the dashboard before it could plausibly handle non-synthetic data.
- Add a human-in-the-loop review workflow: when the model flags a high-risk submission, give the reviewing staff member a structured way to record their own outcome/override, which becomes labeled feedback data for retraining.

- Add request logging, latency monitoring, and basic rate limiting to the scoring API, and formal integration tests that exercise the deployed API contract end to end (not just the demo UI) as part of CI, directly addressing the failure mode described in Section 6.4.
- Extend the CI pipeline to test both deployment variants (Gradio and FastAPI/Docker) automatically, and to diff deployed configuration against the main branch to catch the kind of drift described in Section 6.5 before it reaches production.

9.4 Analytics

- Extend the feature-launch analysis to a proper sequential or Bayesian experimentation framework for future launches, rather than the retrospective diff-in-diff design used here.
- Add a self-serve metrics layer (e.g. a semantic layer on top of the dbt marts) so a non-technical stakeholder could explore metrics dictionary definitions interactively rather than only reading the generated documentation.

10. Conclusion

MSK Pulse demonstrates the complete cycle a health-tech operations and analytics function is expected to run: quantify an operational problem from first principles, build the infrastructure needed to trust that quantification, and ship an intervention that acts on the single highest-leverage finding — not as three disconnected exercises, but as one coherent, reproducible, and honestly-documented system. The engineering problems encountered along the way, from a genuine cross-library version deadlock to a silent API contract mismatch, are the kind of real-world friction a portfolio project built only in a notebook never surfaces, and resolving them in place is as much a part of this project's value as the analyses and the model themselves.